

Agentic AI for Automated Literature Review

Detailed Execution Plan, Phase Roadmap, Team Allocation & Completion Criteria

Project focus: Multi-paper analysis, comparison, research-gap identification and citation-aware literature synthesis.

Purpose

This roadmap converts the project requirements into an executable engineering plan for a 2–3 member team. It defines phases, dependencies, deliverables, ownership, acceptance criteria, evaluation and research-paper milestones. The 15–20 papers are treated as an **initial testing corpus** for validating multi-paper synthesis and gap identification, not as a fixed upper limit for the eventual system.

Current architectural direction

Layer	Proposed choice	Status / rationale
Frontend	Next.js	Planned UI for upload, processing status and results
Backend	FastAPI	Planned Python API/service layer
Agent orchestration	LangGraph	Selected for stateful, explicit, modular workflow control
PDF processing	PyMuPDF initially	Initial candidate; GROBID/Marker can be evaluated if needed
Database	PostgreSQL	Structured storage for papers, summaries, citations and reports
Vector retrieval	pgvector	Semantic retrieval alongside relational data
Validation	Pydantic / structured JSON	Consistent agent inputs and outputs
LLM	Local LLM or API-based LLM	Not finalized; benchmark after hardware/model evaluation

1. End-to-End System Roadmap

The system will be developed as a controlled pipeline rather than as a single large LLM prompt. Individual papers are parsed and summarized first; the resulting structured artifacts are then used for cross-paper comparison, research-gap analysis, citation verification and final report generation.

Core flow

15–20 testing papers → PDF parsing → structured paper objects → individual summaries → multi-paper comparison → research-gap identification → citation/evidence verification → report generation → evaluation

MVP boundary

The first meaningful MVP is reached when the system can take the initial testing corpus and produce structured summaries, a comparison matrix, evidence-supported potential research gaps, traceable source references and a draft literature review.

Dependency chain

Project setup → corpus & parsing → data/retrieval layer → individual agents → LangGraph integration → comparison → gap analysis → citation/evidence layer → report generation → UI → evaluation → final integration

Phase 0 — Project Setup & Architecture

Duration: 2–3 days

Tasks

- Create shared GitHub repository and branching convention.
- Create common Python environment and dependency management.
- Create project folder structure and environment configuration.
- Set up PostgreSQL and verify pgvector availability.
- Define initial API boundaries and LangGraph state model.
- Create README with setup and run instructions.

Deliverables

- Reproducible development environment
- Repository + architecture skeleton
- Database connectivity

Completion gate: Every team member can clone the repository, install dependencies and run the backend/database successfully.

Phase 1 — Research Corpus & PDF Processing

Duration: 4–5 days

Tasks

- Select one focused research domain for initial validation.
- Collect 15–20 relevant full-text research papers.
- Record title, authors, year, DOI/URL and local file metadata.
- Implement initial PyMuPDF extraction.
- Preserve page numbers and document structure.
- Evaluate GROBID/Marker if extraction quality requires it.

- Create page-aware chunks for downstream retrieval.

Deliverables

- Curated 15–20 paper testing corpus
- Parsed paper files
- Page-aware text chunks
- Parser quality notes

Completion gate: The 15–20 testing papers can be parsed with usable text and page references, with failures documented.

Phase 2 — Database & Retrieval Layer

Duration: 3–4 days

Tasks

- Design PostgreSQL tables for papers, sections, chunks, summaries, comparisons, gaps, citations and reports.
- Store parsed content and metadata.
- Generate embeddings for chunks using a selected embedding approach.
- Store vectors in pgvector.
- Implement semantic retrieval with paper/page/section metadata.
- Create retrieval tests.

Deliverables

- Working PostgreSQL schema
- pgvector retrieval
- Source-aware retrieved chunks

Completion gate: An agent/query can retrieve relevant chunks together with paper, page and section information.

Phase 3 — Parsing / Extraction Agent

Duration: 3–4 days

Tasks

- Define a Pydantic/JSON schema for PaperAnalysis.
- Extract research problem, methodology, dataset, metrics, results, findings, limitations and contributions.
- Attach source/page references where possible.
- Validate structured outputs.
- Test across the initial corpus and log extraction failures.

Deliverables

- Structured PaperAnalysis objects
- Validation rules
- Extraction test results

Completion gate: Papers can be converted from parsed content into consistent structured analysis objects.

Phase 4 — Summarization Agent

Duration: 3 days

Tasks

- Define a standardized summary schema.
- Generate concise summaries from structured paper analysis.
- Keep problem, approach, dataset, results, contribution and limitations consistent across papers.
- Store summaries in PostgreSQL.
- Perform manual spot checks against source papers.

Deliverables

- 15–20 standardized summaries
- Summary schema
- Spot-check report

Completion gate: Each testing paper has a usable summary in the same schema.

Phase 5 — LangGraph Workflow Integration

Duration: 3–4 days

Tasks

- Define shared ResearchState.
- Connect Parsing → Summarization nodes.
- Add error handling and status tracking.
- Add conditional routing where needed.
- Prepare interfaces for Comparison, Gap, Citation and Report stages.
- Test complete state propagation.

Deliverables

- Working LangGraph graph
- Shared state model
- Agent transition tests

Completion gate: A corpus processing job can execute implemented stages through a single controlled workflow.

Phase 6 — Multi-Paper Comparison Agent

Duration: 3–4 days

Tasks

- Define comparison dimensions: problem, methodology, dataset, metrics, results, findings and limitations.
- Aggregate individual paper outputs.
- Generate structured comparison rows.
- Handle incompatible or missing metrics without forcing invalid rankings.
- Generate a comparison matrix for the UI/report.

Deliverables

- Structured comparison matrix

- Comparison-table generation
- Comparison validation examples

Completion gate: Multiple paper summaries automatically produce a consistent comparison table.

Phase 7 — Research Gap Identification Agent

Duration: 3–4 days

Tasks

- Use summaries, limitations, findings and comparison results as inputs.
- Identify recurring limitations and unresolved problems.
- Cluster recurring themes.
- Generate potential research gaps.
- Attach supporting papers and evidence to each gap.
- Avoid unsupported generic future-work statements.

Deliverables

- Research-gap objects
- Supporting-paper links
- Evidence-backed gap examples

Completion gate: Each generated potential gap has identifiable supporting papers/evidence.

Phase 8 — Citation & Evidence Verification

Duration: 4–5 days

Tasks

- Map important generated claims to source chunks.
- Store paper/page/section information.
- Retrieve supporting evidence for claims.
- Classify claims as supported, unsupported or requiring re-check.
- Route weak evidence for re-retrieval/review.
- Prepare citation-accuracy evaluation data.

Deliverables

- Claim-to-source mapping
- Evidence records
- Citation verification pipeline

Completion gate: Important generated claims can be traced back to source paper/page/section and verification status is available.

Phase 9 — Report Generation Agent

Duration: 3–4 days

Tasks

- Define literature-review report structure.

- Synthesize verified summaries, comparison results and research gaps.
- Generate coherent sections with citations.
- Keep unsupported claims out or flag them.
- Create exportable report output.

Deliverables

- Draft literature review
- References/citation mapping
- Exportable report

Completion gate: The processed corpus can produce a coherent, source-traceable literature-review draft.

Phase 10 — Frontend & Product Integration

Duration: 4–5 days

Tasks

- Build upload interface.
- Show corpus and processing status.
- Display paper summaries.
- Display comparison tables.
- Display research gaps with supporting papers.
- Display citation/source evidence.
- Display and export final literature review.

Deliverables

- Working web interface
- End-to-end API integration
- Result visualizations

Completion gate: A user can upload papers and navigate from processing to final literature-review output.

Phase 11 — Evaluation & Baseline

Duration: 4–5 days

Tasks

- Define evaluation dataset/reference summaries where applicable.
- Evaluate summarization using ROUGE and BERTScore.
- Evaluate citation accuracy.
- Conduct human evaluation for summary, comparison, gap and report quality.
- Compare against a single-prompt LLM baseline.
- Record errors and failure cases.

Deliverables

- Evaluation report
- Baseline comparison
- Metrics and qualitative analysis

Completion gate: The system has reproducible evaluation results and documented failure cases.

Phase 12 — Research Paper & Documentation

Duration: Parallel; final consolidation in Week 6+

Tasks

- Write introduction/problem definition early.
- Build related-work section alongside corpus study.
- Document methodology, architecture and agents during implementation.
- Document experiments, metrics and baseline.
- Write results, discussion, limitations, conclusion and future work.
- Maintain diagrams, tables and references.

Deliverables

- Complete research-paper draft
- Technical documentation
- Experiment appendix/results

Completion gate: The research paper contains reproducible methodology, experiments, results and discussion.

Phase 13 — Final Integration, Hardening & Demo

Duration: 3–4 days

Tasks

- Run end-to-end tests on 5, 10, 15 and 20 papers.
- Test different PDF layouts and missing metadata.
- Test duplicate papers and parser failures.
- Fix workflow, UI and citation bugs.
- Prepare demo dataset and presentation.
- Tag a stable release.

Deliverables

- Stable prototype
- Demo workflow
- Final documentation
- Release candidate

Completion gate: The complete system runs from upload to literature-review export with known failure modes documented.

2. Team Allocation for 3 Members

Member	Primary ownership	Main phases	Secondary responsibilities
Member 1 — Data & Retrieval	PDF parsing, corpus, PostgreSQL, pgvector, embeddings, source metadata	0, 1, 2, 8	Supports agent inputs and citation evidence
Member 2 — Agentic AI	LangGraph, shared state, Parsing, Summarization, Comparison, Research Gap, Report agents	3, 4, 5, 6, 7, 9	Agent-level testing and workflow reliability
Member 3 — Product & Evaluation	FastAPI, Next.js/UI, API integration, evaluation dashboard, testing, final integration	0, 10, 11, 13	Documentation and demo preparation

Team rule: Each member owns a layer, but integration is shared. All three members should review agent contracts, database schema and evaluation definitions before the first integration milestone.

3. Six-Week Practical Schedule

Week	Primary target	Member 1	Member 2	Member 3	Paper milestone
1	Foundation + testing corpus	PDF/parser + DB	LangGraph skeleton	FastAPI/UI skeleton	Problem definition + initial related work
2	Data + individual agents	Retrieval layer	Parsing + Summarization	APIs + upload UI	Related work + methodology draft
3	Multi-agent synthesis	Data support	Comparison + Gap agents	Integration/UI	Architecture + implementation documentation
4	Citation + report	Evidence layer	Citation + Report agents	Report UI/export	Experimental setup
5	Evaluation	Evaluation data	Agent/error analysis	Evaluation dashboard/tests	Results + discussion
6	Final integration	Hardening	Workflow hardening	Final UI/demo/docs	Complete paper draft

4. GitHub, Branching & Integration Rules

Recommended branch model

main = stable/demo-ready code

develop = integration branch

feature/<name>-<task> = individual work

Definition of Done for every feature

- Code runs locally from a clean environment.
- Inputs and outputs are documented.
- Errors are handled rather than silently ignored.
- At least one automated or reproducible test exists.
- Changes are committed to a feature branch and reviewed before merging.
- Database schema changes are documented.
- Agent outputs use the agreed structured schema.

5. Evaluation Design

Area	Measure	What to test
Summarization	ROUGE	Overlap with reference summaries where reference data is available
Summarization	BERTScore	Semantic similarity to reference summaries
Citation	Citation accuracy	Whether generated claims are supported by cited source evidence
Human quality	Human evaluation	Relevance, completeness, comparison quality, gap quality, coherence
System comparison	Single-prompt baseline	Whether the multi-agent workflow improves selected quality measures

6. Key Technical Decisions Still Open

- **LLM**: not finalized; benchmark local versus API-based options after checking available hardware, context requirements, quality and cost.
- **Research domain**: not finalized; select a focused domain suitable for an initial 15–20 paper testing corpus.
- **PDF parser**: PyMuPDF is the initial candidate; validate it on the actual corpus and use GROBID/Marker if they provide better academic-structure extraction.
- **Embedding model**: select after retrieval-quality and hardware testing.

7. Risk Register

Risk	Impact	Mitigation
PDF extraction errors	High	Preserve pages; test multiple parsers; log failed papers; use fallback parser when justified.
LLM quality insufficient locally	High	Benchmark local models early; keep API option available.
Long context / many papers	High	Chunk + retrieve relevant evidence; avoid passing the full corpus to every agent.
Unsupported research gaps	High	Require supporting papers/evidence for generated gaps.

Risk	Impact	Mitigation
Citation errors	High	Claim-to-source mapping + verification + citation accuracy evaluation.
Agent workflow failures	Medium	Typed state, validation, retries and explicit conditional edges.
Team merge conflicts	Medium	Feature branches, shared interfaces and frequent integration.

8. Final Acceptance Checklist

- 15–20 initial testing papers can be uploaded and parsed.
- Paper metadata, page numbers and chunks are preserved.
- Individual paper analysis and summaries are generated in a consistent schema.
- Multiple papers can be compared automatically.
- Comparison output can be rendered as a structured table.
- Potential research gaps include supporting papers/evidence.
- Important generated claims have source/page/section traceability.
- An end-to-end literature-review draft can be generated.
- Frontend can display the major outputs.
- Evaluation results and failure cases are documented.
- Known limitations are recorded.
- Research paper draft contains methodology, experiments, results and discussion.

9. Daily Team Operating Rhythm

Start of day: 10-minute sync — completed work, today's task, blockers.

End of day: push code + short progress note + tests/results.

Every 2–3 days: integrate to develop and run a smoke test.

End of each phase: demonstrate the completion gate before starting dependent work.

10. Immediate Next Actions

The first implementation sprint should focus on the foundation: create the shared repository and environment, decide the initial domain, collect the 15–20 paper testing corpus, test PDF extraction quality, establish PostgreSQL/pgvector, and create the LangGraph skeleton. Do not spend the first sprint polishing the UI or prematurely locking the LLM.

Execution principle: Build the smallest end-to-end path early, then improve each layer using measurable tests. This prevents the team from spending weeks building isolated components that do not integrate.